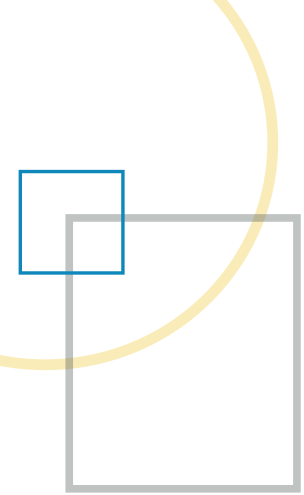
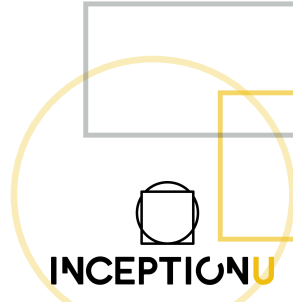




Evolve Full Stack Developer

TypeScript



Agenda

- What is TypeScript?
 - Introduction to Types and Syntax
 - Working with TypeScript
-
- Join Menti Meter: <https://www.menti.com/alfynh5omgvf>
 - Examples on <https://github.com/EvolveU-FSD/c10-typescript>

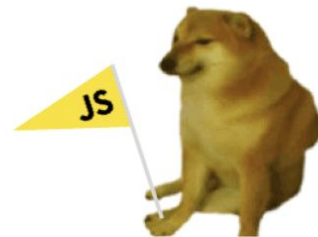


What is TypeScript?



TypeScript

- TypeScript is a **strongly typed** programming language that builds on JavaScript
- It is a **superset** of JavaScript



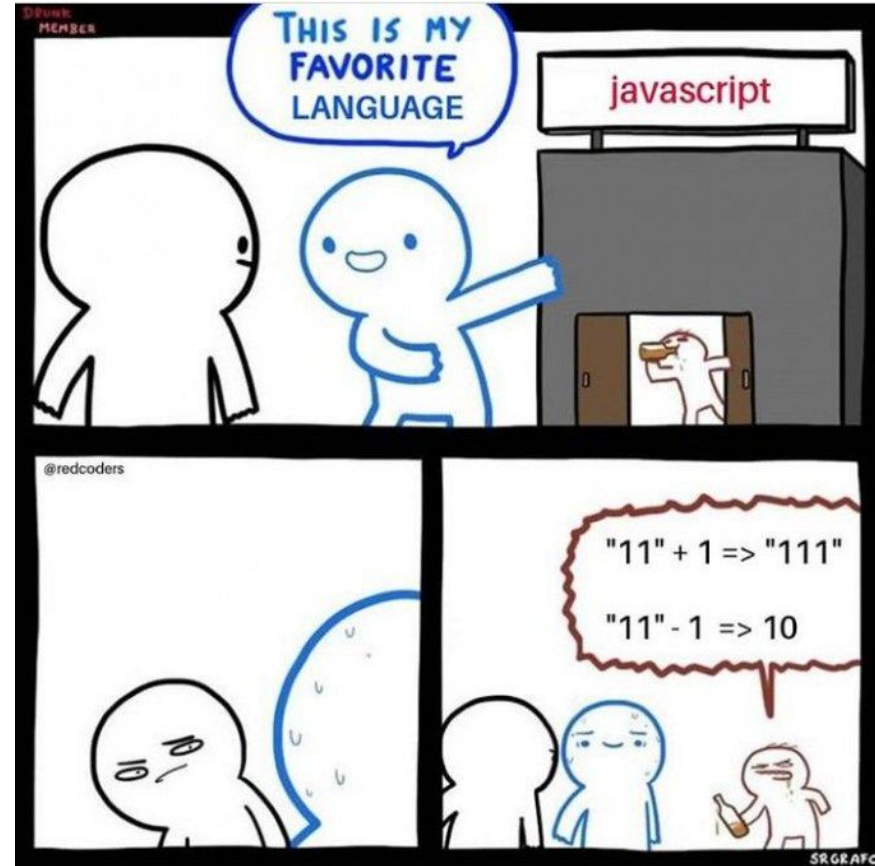
Why might this be a good thing?

(See Menti Meter Slide)



Why is TypeScript?

- Detect bugs before you run the code
- Prevents “shooting yourself in the foot”
- Predictable code
- Better developer experience



Types and Syntax



Declarations

```
let isDone: boolean;
```

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

```
let cheese:string = "mozerella";
```



Basic Types

any

boolean

number

string

null

undefined

string[] /* or Array<string> */

[string, number] /* tuple */

string | null | undefined /* union */



Type Assertion

- A way to override type defaults
- Handy when converting JS to TS

JS

```
let food = {};  
food.price = 5;  
food.name = 'hamburger';
```

TS

```
interface Food {  
  price: number;  
  name: string;  
}  
var food = {} as Food;  
food.price = 5;  
food.name = 'hamburger';
```



Interfaces

- interfaces in TypeScript are open ended
- Allows us to mimic the extensibility of JavaScript

```
// Example 1
declare var fruit1: { howMany: number; colour: string; whoEats: string[]; };

// Example 2
interface Fruit {
  howMany: number; colour: string; whoEats: string[];
}
declare var fruit2: Fruit;
```



Optional Properties

- Helps dealing with objects that might have a property set

```
interface User {  
  name: string;  
  age?: number;  
}  
  
function printName(user: User, includeMiddle?: boolean) {  
  ...  
}
```



Type Aliases

- defining types with a custom name
- used for primitives (like string or complex types like objects and arrays)

```
type BossName = string;
type Attacks = string[];
type Health = number;
type SoulsDropped = number;
type ItemDrops = string[];

type BossFight = {
  name: BossName,
  attacks: Attacks,
  health: Health,
  souls: SoulsDropped,
  items?: ItemDrops
}
```



Function Types

- Specify the return type and the parameter types

```
Example 1: Function Returning a Number
function getTime(): number {
    return new Date().getTime();
}

// Example 2: Function returning nothing
function bahHumBug(): void {
    console.log('Go away!');
}

// Example 3: Function returning a number with number parameters
function addMyNumbers(firstNumber: number, secondNumber: number): number {
    return firstNumber + secondNumber;
}

let sum = (firstNumber: number, secondNumber: number): number => {
    return firstNumber + secondNumber;
}
```



Type Inference

- TypeScript knows the JavaScript language and will generate types for you (in many cases)

```
const favouriteColour = "red";
```

```
const favouriteColour: string
```

```
function add(a: number, b: number): number
```

```
function add(a: number, b: number) {  
  return a + b;  
}
```

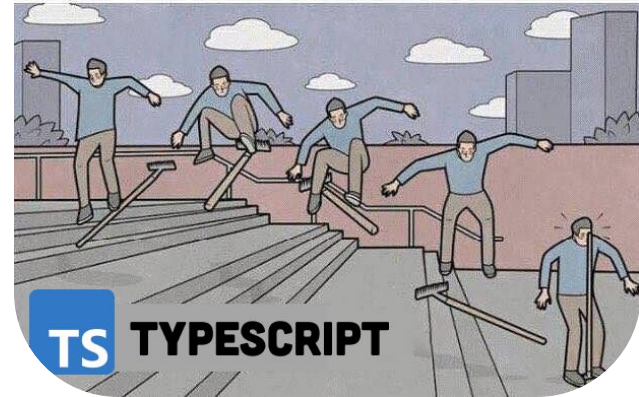
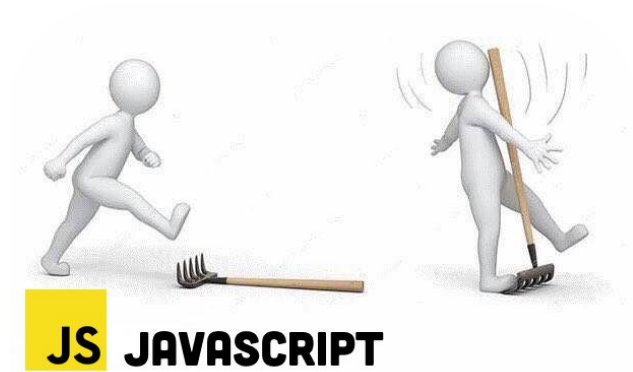
Working with TypeScript

JS devs when we start using
typescript



Installation & Compilation (transpilation)

- `npm install -g typescript`
- `tsc <the name of your file>.ts`
- .js file will be generated



How will it Compile?

(See Menti Meter Slide)



tsconfig.json

- Lives in the root directory of your project
- Create this file by running “tsc --init”
- Running the command “tsc” will reference this file
- “tsc -w” will watch for changes and recompile
- Can include and exclude files from transpilation



React Implications

- .jsx becomes .tsx
- Use "jsx" : "react" in your tsconfig.json's compilerOptions.
- Install the definitions for JSX and React into your project : `(npm i -D @types/react @types/react-dom).`



JS vs TS - What tests would you need to write?

(See Menti Meter Slide)

JS

```
function addMyNumbers(firstNumber, secondNumber) {  
  return firstNumber + secondNumber;  
}
```

TS

```
function addMyNumbers(firstNumber: number, secondNumber: number): number {  
  return firstNumber + secondNumber;  
}
```



Other Implications

- New build step “npx tsc -w & npx nodemon build”
- js files should be added to the .gitignore
- Fewer tests to write



Resources

- <https://basarat.gitbook.io/typescript/getting-started>
- <https://www.typescriptlang.org/>
- <https://www.w3schools.com/typescript/index.php>
- <https://www.tutorialsteacher.com/typescript/>

